COLLEGE ADMISSION PROCESS

Functional Requirements & Claude Code Implementation Guide

Technology context: Existing Next.js frontend + Laravel backend college / higher-education ERP. This document defines a configurable baseline; Claude Code must inspect the repository and adapt it to existing architecture, naming, permissions, APIs, and database conventions.

1. Purpose and Expected Outcome

Build or improve a complete College Admission Module that manages the journey from initial inquiry through application, eligibility/document verification, selection or offer, fee payment, admission confirmation, and creation/linking of the student record. The module must be secure, auditable, configurable by academic year/program/department, and integrated with existing ERP modules without breaking current workflows.

- Use the current project architecture and coding patterns; do not rebuild the application or introduce a parallel framework.
- Do not assume database table names, routes, role names, or menu labels before inspecting the repository.
- Reuse existing authentication, role/permission checks, academic masters, student records, fee structures, and file-upload utilities wherever possible.
- If a feature is not supported by existing business rules, document the assumption and implement it as a configurable setting where practical.

2. Admission Lifecycle — End-to-End

Stage	What happens	Possible status
1. Admission Inquiry	Capture prospective student's contact details, interested program, academic year, source, and follow-up notes.	New, Contacted, Follow-up, Converted, Closed
2. Application Registration	Create an application number and capture applicant profile, contact details, program choice, and previous education.	Draft, Submitted
3. Document Upload	Applicant/staff uploads required certificates and identity/supporting documents.	Pending, Uploaded, Verified, Rejected, Resubmission Required
4. Eligibility Review	Check program-specific qualification, marks/grades, required subjects, and configured criteria.	Pending Review, Eligible, Ineligible, Needs Review
5. Application Review / Merit	Review application, enter verified marks where required, apply configured merit/selection rules, and record decision.	Under Review, Shortlisted, Waitlisted, Rejected, Selected
6. Offer / Seat Allocation	Issue provisional offer, select program/department/seat category where applicable, and record response deadline.	Offer Issued, Accepted, Declined, Expired
7. Admission Fee	Generate applicable admission/first-installment demand using the existing fee engine; record payment through the existing payment workflow.	Unpaid, Partially Paid, Paid, Failed, Waived
8. Admission Confirmation	Confirm only after required checks are complete or an authorized override is recorded.	Confirmed, On Hold, Cancelled

9. Student Record Creation	Create or link the student record, assign student ID/roll number as per existing rules, and link application, program, academic year, and fee records.	Created / Linked
10. Post-Admission	Make the student visible in appropriate ERP modules and provide confirmation/receipt/next-step information.	Active, Pending Onboarding

3. Menu Structure and Navigation

Adapt these labels to the current dynamic menu and permission system. Do not create duplicate menus if equivalent screens already exist.

Parent menu	Submenu / screen	Primary actions
Admission	Admission Dashboard	View counts by status, recent applications, pending verification, pending fees, and quick actions.
Admission	Admission Inquiry	Create, search, view, edit, assign follow-up, convert to application, close inquiry.
Admission	Application Registration	Create application, save draft, submit, search, view application timeline.
Admission	Application Review	Review details, check eligibility, verify documents, add internal notes, record decision.
Admission	Document Verification	Filter pending/rejected documents, preview/download securely, verify or request resubmission.
Admission	Merit / Selection	View eligible applicants, apply approved selection rules, record merit/selection decision where applicable.
Admission	Offer / Seat Allocation	Issue offer, record acceptance/decline, assign program/seat details if applicable.
Admission	Admission Fee	Generate/view fee demand, see payment status, link to existing payment/receipt screens.
Admission	Admission Confirmation	Run confirmation checklist, confirm/hold/cancel with permission and audit trail.
Admission	Admitted Students	Search confirmed admissions, view linked student profile, export permitted reports.
Admission	Reports	Inquiry funnel, application status, pending documents, eligibility/selection, fee status, confirmed admissions.
Admission	Admission Settings	Academic-year intake, program requirements, required documents, status rules, numbering formats, deadlines, and configurable fees where authorized.

4. Role and Permission Matrix

Map these responsibilities to roles already present in the application. If the ERP uses different role names, preserve the existing role system and permission conventions.

Role / responsibility	Typical permissions
System / ERP Admin	Configure admission settings, academic year and program intake; manage access; view all records permitted by institutional scope; authorize exceptional overrides.
Admission Officer	Manage inquiries and applications; update applicant details where allowed; review eligibility; verify documents; record decisions; prepare offers.
Accounts / Fee Operator	View fee demands and payments; process or reconcile payments through the existing finance workflow; issue receipts according to existing permissions. Cannot independently approve academic eligibility.
Admission Approver / Committee	Review shortlisted applications and record selection/approval decisions; access only the data required for the assigned scope.
Applicant / Student (only if applicant portal exists or is in scope)	Create/update own application before submission, upload documents, view application status, respond to offer, and view payment/confirmation information.
Teacher / Other ERP User	No admission access by default unless explicitly granted through existing permissions.

- Enforce permissions on the Laravel backend, not only by hiding frontend menu items.
- Scope records by institution/campus/branch/department/academic year if the existing ERP supports these boundaries.
- Log important actions with user, timestamp, action, record ID, and before/after values where appropriate.

5. Screen-Level Functional Requirements

5.1 Admission Dashboard

- Show total inquiries, draft/submitted applications, pending document verification, pending eligibility review, offers awaiting response, pending fees, and confirmed admissions.
- Allow authorized users to click a metric and drill down to a filtered list.
- Respect permissions and institution/branch/program scope in every count and drill-down.
- Include loading, empty, error, and unauthorized states.

5.2 Admission Inquiry

- Fields: inquiry number, applicant name, phone, email, inquiry date, academic year, interested program/course, campus/branch if relevant, source, assigned officer, follow-up date, remarks, status.
- Validate required fields and phone/email formats; avoid duplicate inquiries using configurable matching rules.
- Support search, filters, pagination, sorting, and status history.
- Provide a controlled action to convert an inquiry to an application without duplicating the applicant unnecessarily.

5.3 Application Registration

- Generate a unique application number on the backend; never rely on a client-generated number for uniqueness.
- Capture personal/contact information, address, program preferences, academic history, applicable category/reservation fields if institutionally required, and emergency/guardian details if required.
- Support draft save and final submission. After submission, restrict edits to authorized correction/reopen workflows.
- Display a clear checklist of missing required fields and documents before submission.

- Prevent accidental duplicate submissions and preserve the application audit history.

5.4 Documents

- Required documents must be configurable by program, admission type, academic year, and applicant category where relevant.
- Store file metadata and secure storage references; validate allowed file types and size; do not expose private files through public URLs.
- Track each document independently: pending, uploaded, verified, rejected, resubmission required.
- A rejection must include a reason visible to the appropriate user.
- Record who verified/rejected a document and when.

5.5 Eligibility and Selection

- Eligibility rules must be configurable and based on approved institutional criteria; do not hard-code arbitrary marks or cutoffs.
- Show the values used in the eligibility decision and a clear reason for eligible/ineligible/needs-review status.
- If merit ranking is required, document the approved calculation and tie-break rules before implementing them.
- Manual overrides require a permission, reason, timestamp, and audit record.
- Do not automatically reject applicants because a field is missing or ambiguous; route them to Needs Review when appropriate.

5.6 Offer and Seat Allocation

- Issue an offer only to an applicant whose status permits it under configured workflow rules.
- Capture offer date, expiry/deadline, program/seat details where applicable, decision, and notes.
- Prevent duplicate active offers for the same application unless the business rules explicitly allow them.
- Track accepted, declined, and expired offers with history.

5.7 Fees and Payment

- Integrate with the existing ERP fee module and payment gateway. Do not create a separate payment ledger if one already exists.
- Define which charges are due at each admission stage; amount and due date should come from approved fee configuration.
- Handle pending, successful, failed, partially paid, refunded, and waived states only as supported by the existing payment system.
- Use server-side payment verification/webhooks where applicable. Never mark a payment successful based only on a frontend response.
- Generate or link receipts using the existing receipt process.
- Define whether fee payment is mandatory for confirmation and support an authorized waiver/hold path if institutional policy allows it.

5.8 Admission Confirmation and Student Record

- Show a pre-confirmation checklist: application submitted, required documents verified, eligibility/selection approved, offer accepted if applicable, required fee condition met, and required program/academic-year data available.
- Block confirmation if required conditions are incomplete, unless a permitted override with a mandatory reason is recorded.
- Confirmation must be idempotent: repeated clicks or retries must not create duplicate student records, enrollments, or fee demands.
- Create or link the student record using the existing student creation service and numbering rules.
- Link the student record to the admission application, selected program, academic year, campus/branch, and fee/payment records as supported.
- Use a database transaction for the critical confirmation operation and handle failures safely.

- After success, show the generated/linked student identifier and next steps. Make the student visible to the correct ERP modules according to existing activation rules.

6. Suggested Data Model — Validate Against Existing Schema First

These are conceptual entities, not instructions to create tables blindly. Inspect migrations and models first; reuse existing entities and add only the missing fields/tables.

Conceptual entity	Example data
AdmissionInquiry	id, inquiry_number, applicant_name, phone, email, academic_year_id, program_id, source, assigned_to, follow_up_at, status, notes, created_by, timestamps
AdmissionApplication	id, application_number, inquiry_id, applicant/user reference, academic_year_id, program_id, personal/contact data or profile reference, previous_education, submitted_at, status, decision fields, timestamps
AdmissionDocument	id, application_id, document_type_id, storage_path, original_filename, mime_type, size, verification_status, rejection_reason, verified_by, verified_at
AdmissionReview / History	id, application_id, action/status, comments, actor_id, created_at; preserve important state transitions
AdmissionOffer	id, application_id, offer_date, expiry_date, offer_status, program/seat references, accepted_at
Admission-to-Student Link	application_id, student_id, confirmed_at, confirmed_by; enforce uniqueness according to business rules
Admission Settings	academic year/intake, program requirements, document rules, deadlines, numbering and approved workflow settings

- Use the project's existing primary-key types, naming conventions, soft-delete conventions, tenant/branch scoping, and timestamps.
- Add foreign keys and indexes for common filters and relationships where consistent with the current database.
- Use unique constraints for application numbers and other identifiers that must be unique.
- Do not store raw card data or sensitive payment credentials.
- Do not duplicate student master data if a student/applicant profile already exists; link records where appropriate.

7. API and Backend Expectations

Follow existing Laravel route grouping, controller/service conventions, API response format, authentication middleware, and validation patterns. Names below are illustrative; adapt them to the repository.

Operation	Illustrative endpoint/action
Inquiry list/create/detail/update	GET/POST /admission/inquiries; GET/PUT /admission/inquiries/{id}
Convert inquiry	POST /admission/inquiries/{id}/convert
Application list/create/detail/update	GET/POST /admission/applications; GET/PUT /admission/applications/{id}
Submit application	POST /admission/applications/{id}/submit
Upload document	POST /admission/applications/{id}/documents
Verify/reject document	POST /admission/documents/{id}/verify or /reject
Eligibility/review decision	POST /admission/applications/{id}/eligibility or /review
Offer decision	POST /admission/applications/{id}/offer
Fee demand/payment status	Use existing fee/payment APIs; add only the missing integration
Confirm admission	POST /admission/applications/{id}/confirm
Dashboard/reports	GET /admission/dashboard; GET

- Validate every request on the backend and authorize each action against the target record.
- Use consistent status codes and the project's standard JSON envelope.
- Add pagination and server-side filtering for list endpoints.
- Prevent mass assignment of protected fields such as status, student ID, payment status, and confirmation metadata.
- Use transactions for multi-step critical operations and idempotency protections for conversion, payment callbacks, and confirmation.
- Avoid N+1 queries; eager-load required relations and add indexes only after checking actual query patterns.

8. Frontend Requirements (Next.js)

- Use the existing App Router/layout conventions, design system, form components, tables, modal patterns, and API client.
- Create responsive screens with clear page titles, breadcrumbs, status badges, search, filters, pagination, and action menus.
- Use field-level validation messages and a summary of errors on submit.
- Confirm destructive or irreversible actions such as reject, cancel, and confirm admission.
- Disable duplicate submissions while requests are in progress and display actionable success/error messages.
- Render menus and actions from the existing role/permission system; backend authorization remains authoritative.
- Provide loading skeletons/spinners, empty states, retry states, and clear unavailable/unauthorized messages.
- Do not replace the current visual design system or introduce new UI libraries without a demonstrated need.

9. Reports and Exports

- Inquiry funnel by source, program, campus/branch, and date range.
- Applications by status, program, academic year, and assigned officer.
- Pending document verification and resubmission list.
- Eligibility/selection/merit report, where applicable.
- Offers issued, accepted, declined, and expired.
- Admission fee due/paid/pending report based on the existing fee ledger.
- Confirmed admissions and student IDs.
- Exports must respect the same authorization and scope as the screen. Follow existing CSV/Excel/PDF export patterns.

10. Validation, Security, and Audit

- Authentication and authorization required for all protected APIs and files.
- Validate uploads by MIME/type, extension, size, and secure storage rules.
- Protect against duplicate records, invalid status transitions, unauthorized edits, and cross-branch data access.
- Keep a state-transition history for submission, verification, eligibility, offer, payment linkage, confirmation, cancellation, and overrides.
- Do not log passwords, access tokens, full payment credentials, or unnecessary sensitive applicant data.
- Use soft deletes or cancellation states where the current ERP's audit policy requires retaining historical records.
- Apply privacy and retention requirements relevant to the institution and jurisdiction.

11. Acceptance Criteria

- Authorized users can create, search, filter, view, and update inquiries.

- An inquiry can be converted into one application without duplicate applicant/application records.
- Applications support draft and submit states with clear validation.
- Required documents are configurable and can be uploaded, verified, rejected, and resubmitted with history.
- Eligibility and selection follow configured/approved rules and explain the recorded decision.
- Offer and response statuses are tracked correctly.
- Fee information comes from the existing fee/payment system and cannot be falsely marked paid from the client.
- Admission confirmation checks all required conditions and records any authorized override.
- Confirmation creates or links exactly one student record and does not duplicate it on retry.
- Menus, screens, APIs, and reports respect role permissions and institution/branch scope.
- Dashboard metrics match the underlying filtered records.
- Existing modules, login, menus, APIs, and student workflows continue to work.
- Automated tests cover core workflows, permission failures, duplicate prevention, validation, and confirmation retry behavior.

12. Test Scenarios

Test	Expected result
Valid inquiry creation	Inquiry saved and searchable with generated unique number.
Duplicate inquiry/application attempt	Configured duplicate warning or safe prevention; no accidental duplicate conversion.
Missing required application fields	Submission blocked with field-level validation.
Invalid document type/oversized file	Upload rejected securely with a clear message.
Document rejected	Reason saved; status changes to resubmission required; history recorded.
Ineligible application	Cannot proceed to offer unless an authorized, audited override is permitted.
Unauthorized user confirms admission	Backend returns an authorization error; no data changes.
Fee payment pending/failed	Confirmation blocked if policy requires successful payment.
Repeated confirmation request	Same student link/result returned or safe conflict handled; no duplicate records.
Failure during confirmation transaction	Partial records are rolled back or safely recoverable; application remains consistent.
Cross-branch access attempt	User cannot view or modify out-of-scope records.
Dashboard drill-down	Metric opens the matching filtered list and count matches the data.
Regression	Existing login, dynamic menus, student management, fees, and reports remain functional.

13. Recommended Implementation Plan

1. Phase 0 — Repository audit: inspect the folder structure, current admission/student/fee modules, migrations, models, controllers, services, routes, API client, dynamic menus, permissions, and tests. Report what already exists and what is missing.
2. Phase 1 — Workflow and schema mapping: map this document to real tables and status enums. Identify required decisions/assumptions before making schema changes.
3. Phase 2 — Backend foundation: migrations only for missing fields/entities, models/relationships, validation, authorization, services, routes, audit history, and automated tests.
4. Phase 3 — Inquiry and application UI: dashboard, inquiry list/form/detail, conversion, application registration, draft/submit, and document upload.
5. Phase 4 — Review workflow: document verification, eligibility, review/selection, offer and response handling.
6. Phase 5 — Fees and confirmation: integrate existing fee/payment modules; implement safe idempotent confirmation and student linking.

7. Phase 6 — Reports and permissions: dashboard drill-downs, reports/exports, menu rights, branch/program scoping.
8. Phase 7 — Verification: run relevant backend tests, frontend lint/typecheck/build, migration checks, and regression tests; report exact results.

14. Mandatory Instructions for Claude Code

You are working in an existing College / Higher Education ERP with a Next.js frontend and Laravel backend.

Implement the College Admission Module according to this document, but FIRST inspect the repository. Do not start by blindly creating files or tables.

MANDATORY RULES
1. Audit the current codebase, existing admission-related screens, student master, academic-year/program/department masters, fee and payment workflow, authentication, roles/permissions, dynamic menu system, API patterns, database migrations, and tests.
2. Return a concise audit: existing functionality, missing functionality, reusable components/services, relevant files, and a phased plan. Identify any business-rule ambiguity instead of inventing institutional rules.
3. Preserve the current architecture, naming conventions, design system, response formats, and established patterns. Do not rewrite unrelated modules.
4. Map every requirement in this document to existing code or a proposed change. Reuse existing student and fee records; do not create duplicate ledgers or student masters.
5. Treat endpoint names and conceptual table names in this document as examples only. Confirm actual conventions before implementation.
6. Implement in small, reviewable phases. After each phase, run relevant tests and type/lint checks, inspect the diff, and report files changed and results.
7. Enforce authorization and data scope in Laravel APIs/services, not just in the frontend. Respect institution/campus/branch/program scope if present.
8. Make conversion, payment callbacks, and admission confirmation safe against retries and duplicate records. Use database transactions for critical multi-record operations.
9. Do not invent eligibility thresholds, merit formulas, fee amounts, required documents, or payment/confirmation policies. Reuse configured institutional rules or ask for clarification where a decision is essential.
10. Do not delete production data, run destructive database commands, reset databases, or apply migrations to production. Provide migration and rollback instructions and use a safe development/test environment.
11. Do not claim tests/builds passed unless you actually ran them. Report failures and unresolved risks honestly.
12. Keep existing login, menus, student management, fees, and unrelated workflows working.

DELIVERABLES
- Repository audit and gap analysis.
- Updated implementation plan mapped to actual files.
- Database/migration changes only where required.
- Laravel APIs/services/validation/authorization/audit history.
- Next.js screens and navigation integrated with the existing menu/permission system.
- Integration with the existing student and fee/payment modules.
- Automated tests for core flows, permissions, duplicate prevention, and confirmation retry.
- A final summary listing files changed, migrations, routes, test/build commands and actual results, assumptions, and remaining work.

Start with the audit and plan. Do not implement the whole module before presenting the audit and confirming any essential business-rule decisions.

15. Business Decisions to Confirm Before Production

- Which academic years, programs, campuses/branches, and admission types are in scope?

- Is the applicant portal already available, or will staff manage all applications?
- Which documents are mandatory for each program/admission type?
- What eligibility and merit rules have been formally approved?
- Which fee components must be paid before admission confirmation, and are waivers allowed?
- When should the student record become active, and how are student ID/roll number generated?
- Are seat limits, reservation/category rules, waitlists, or multiple program preferences required?
- Which notification channels already exist (email, SMS, in-app), and which admission events should trigger them?

16. Definition of Done

The module is complete only when the end-to-end workflow works using the real project's authentication, database, student records, fee/payment workflow, and dynamic permissions; core and regression tests pass; migrations are reviewed; unauthorized access and duplicate confirmation are tested; and the final implementation report lists actual verification results.